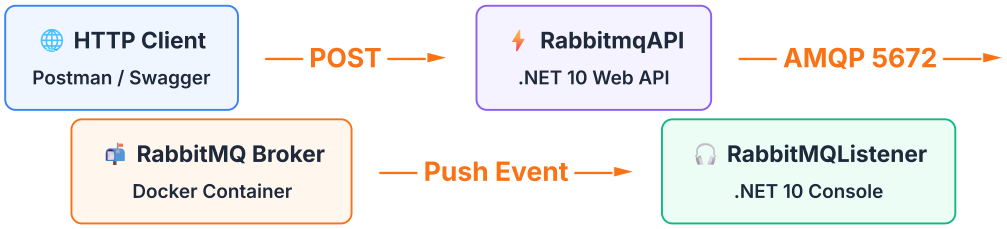
RabbitMQ Message Broker Architecture

Architecture & Reference Manual

Publisher-Subscriber Distributed System (.NET 10 & RabbitMQ 3.x)

1. System Overview

This system implements an asynchronous, decoupled, message-driven architecture. A REST API publisher receives geographic location updates via HTTP and immediately offloads processing to a RabbitMQ message queue. Downstream background workers (listeners) consume and acknowledge messages asynchronously.



2. System Components

🚀 1. Producer: RabbitmqAPI

Path:
`d:\Robin\Pearsonal\Rabitmq\API\RabbitmqAPI`

Framework: ASP.NET Core (.NET 10.0)

Endpoint: `POST /api/Location`

Receives location models, establishes non-blocking async connection to RabbitMQ, verifies the queue, and executes `BasicPublishAsync`.

🎧 2. Consumer: RabbitMQListener

Path:
`d:\Robin\Pearsonal\Rabitmq\Console\RabbitMQListener`

Framework: .NET 10.0 Console Application

Consumer Type: `AsyncEventingBasicConsumer`

Maintains persistent listening loop, decodes binary payload into string objects, and manually issues `BasicAckAsync` delivery confirmation.

📬 3. Message Broker: RabbitMQ Docker Instance

Image: `rabbitmq:3-management` | **Container Name:** `rabbitmq`

AMQP Port (5672): Binary messaging port for clients and listeners.

Management UI Port (15672): Web management dashboard at http://localhost:15672 (Credentials: `guest` / `guest`).

Queue Name: `locationSampleQueue` (Durable: `false`, AutoDelete: `false`, Exclusive: `false`)

3. Message Contract & Data Structure

C# Model Definition:

```
public class Location
{
    public DateTime Date { get; set; }
    public double Latitude { get; set; }
    public double Longitude { get; set; }
}
```

Published String Format:

`"Latitude: 12.9716, Longitude: 77.5946 and Time: 10/08/2026 14:30:00"`

4. End-to-End Workflow & Execution Sequence

Step	Actor / Component	Action / Method	Description
1	**RabbitMQListener**	`CreateConnectionAsync()`	Establishes AMQP connection and channel to RabbitMQ broker on port 5672.
2	**RabbitMQListener**	`BasicQosAsync(prefetch: 1)`	Configures Quality of Service to fair-dispatch 1 message at a time.
3	**Client ➔ API**	`POST /api/Location`	Client submits location JSON to ASP.NET Core API controller.
4	**API ➔ RabbitMQ**	`BasicPublishAsync(...)`	API formats payload bytes and publishes to `locationSampleQueue` via default exchange.
5	**RabbitMQ ➔ Listener**	`ReceivedAsync Event`	RabbitMQ pushes message to listener callback; listener decodes and processes message.
6	**Listener ➔ RabbitMQ**	`BasicAckAsync(DeliveryTag)`	Listener confirms successful processing; broker safely clears message from queue.

5. Key Technical Concepts & Best Practices

Concept	Implementation	Architectural Benefit
Default Exchange (`""`)	Direct routing by queue name	Simplifies 1-to-1 work queues without explicit exchange bindings.
Manual Acknowledgment	`autoAck: false` + `BasicAckAsync`	Guarantees zero message loss if consumer crashes mid-processing.
Fair Dispatch (QoS)	`prefetchCount = 1`	Distributes work evenly across multiple consumer instances.
Async Event Consumer	`AsyncEventingBasicConsumer`	Fully non-blocking asynchronous I/O matching .NET 10 Task-based async pattern.

6. Step-by-Step Operations Guide

1️⃣ Start RabbitMQ Docker Container

```
docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-management
```

If already created: `docker start rabbitmq`

2️⃣ Launch Consumer (RabbitMQListener)

```
dotnet run --project D:\Robin\Pearsonal\Rabitmq\Console\RabbitMQListener\RabbitMQListener\RabbitMQList
```

3️⃣ Launch Publisher (RabbitmqAPI)

```
dotnet run --project D:\Robin\Pearsonal\Rabitmq\API\RabbitmqAPI\RabbitmqAPI\RabbitmqAPI.csproj
```

4️⃣ Send Test Location via cURL / Swagger

```
curl -X POST "https://localhost:7000/api/Location" \
     -H "Content-Type: application/json" \
     -d '{"Latitude": 12.9716, "Longitude": 77.5946, "Date": "2026-10-08T14:30:00Z"}'
```

7. Troubleshooting Guide

> **BrokerUnreachableException (Refused on 127.0.0.1:5672):** RabbitMQ broker is not running. Ensure Docker Desktop is open and run `docker start rabbitmq` .

> **File Locked Error on Build:** An existing instance of `RabbitmqAPI.exe` is running. Stop the process in your terminal or task manager before rebuilding.
